Sports data gameplan (mutually agreed plan)

What this file is: The **forward-looking contract** for what we **will** build and **how** we will do it—**not** a description of legacy notebook behavior. We **implement** in importable **Python modules** under `sports_pipeline/` (see root `pyproject.toml`; optional `pip install -e .`) and orchestrate from `530_sports_pipeline.ipynb` as the **conductor**. **Working copies** of the old multi-notebook flow were removed from the project root; **migration / salvage** code and narrative live only in `obsolete_documents/sports_gameplan_old/` and `obsolete_files/sports_gameplan_old/`.

Lineage: The original narrative came from `../../1-Various_PDE_and_Chat_stuff/4-coding_stuff/SPORTS DATA Flow.docx`. Historical **cell-level** run order for the retired multi-notebook layout is preserved in `obsolete_documents/sports_gameplan_old/Scout_College_Basketball_Notebook_Workflow_How_To_bkup.md`; a **successor** how-to may be added here once the new conductor exists. Until then, treat **this** file as the source of truth for **intent**.

Working agreement (process)

1. **Document first, code second.** We refine **this gameplan** until the flow is right on paper; then we change notebooks to match. Large matcher or **530** rewrites only after we mark sections ready—or add an explicit "approved for implementation" note.
2. `dedupe` **(the package) is optional for *this* pipeline.** We will **not** depend on it to ship the college draft ↔ ESPN spine path. Interest in `dedupe` includes **learning entity resolution** for **later, larger datasets**; the vendored tree `python_packages/dedupe/` supports study and experiments outside the critical path.
3. **Snapshots:** Use `sports_gameplan_old` folders (and similar) so we always know what "current truth" is on disk.

Target architecture: 530 as pipeline conductor

Intent: `530_sports_pipeline.ipynb` (or a renamed successor) is the **single entry point** you run end-to-end. It calls **importable** `.py` **modules** for heavy lifting instead of asking you to hop across `sdv_first` → `sdv_second` → `scrape_bpm` → `bpm_merge` manually.

We will:

Role	Responsibility
`.py` **libraries**	BOX ingest / spine refresh; draft ↔ spine matching + exports; SR scrape helpers; BPM crosswalk + match + job list; shared path constants (`DO_NOT_ERASE` vs working `mbb/`). Functions return DataFrames or write known outputs; minimal I/O hidden inside clear APIs.
`530` **notebook**	High-level **stages in order**: config → optional spine refresh → draft match → human **pause points** (see below) → panel build → `PERF_METRIC` / pool / plots / LPM → export. Thin cells: imports, `run_stage_foo(config)`, print summaries, optional `%run` or **IPython** display.
Human feedback loops	**Explicit breakpoints**—e.g. after draft exports, script **stops** (or a cell says "edit `DO_NOT_ERASE/college_aliases.csv` / feedback template, then re-run from here") until you are satisfied, then **one** "resume" cell continues. Not scattered across four notebooks.

Why this fits the gameplan: It matches **document-first** (stages are explicit), **clean working dirs** (`DO_NOT_ERASE` holds durable inputs; `mbb/` can be wiped and rebuilt), and your preference for a **straight-line** run with a **few** controlled loops instead of notebook-hopping.

Migration: Factor logic from `sports_gameplan_old` snapshots into modules **incrementally** (e.g. `ingest_box.py`, `draft_match.py`, `sr_scrape.py`, `bpm_merge.py`) as the new **530** conductor and libraries come online.

Outcomes we want

- A **reproducible spine** from **BOX** (player / team / schedule) keyed by `athlete_id`, `team_id`, `season`.
- **Draft picks** (with college strings) linked to that spine at `athlete_id` with **clear human-auditable** steps and CSVs for misses.
- A **player–season panel** (**530**) with **draft outcome**, **leave-one-out pool-quality** on a chosen **performance measure**, **EDA** (e.g. binned draft rate vs pool quality), and **baseline regression**—designed so we can swap **PPM / minutes / BPM / OBPM / DBPM** as `PERF_METRIC` once SR merge exists.
- **SR advanced stats** merged onto the **same** ESPN-keyed panel rows—not a second spine.

Durable team / school string equity (CSV)

Intent: The prior pipeline produced **hand-curated** mappings on disk. New implementation **should load and respect these files when present**—they are **not** throwaway cache; regenerating school/slug logic from scratch would waste that work. **Extend** them only when QA artifacts (unmapped colleges, bad slugs, 404 repair lists) show gaps; treat empty or missing files as "build from fuzzy + review," not as a reason to ignore this contract.

Canonical layout (see also `datasets/mbb/DO_NOT_ERASE/README.txt`):

File (conventional location)	Stage	Role
`DO_NOT_ERASE/college_aliases.csv`	2a	Maps draft `college` text → ESPN-side school / `team_short_display_name` (and related) so name matching can enforce **school second**.
`DO_NOT_ERASE/sr_school_slug_crosswalk.csv`	3	Maps ESPN `team_id` → Sports-Reference **school slug** for URLs (**canonical** location; conductor may still fall back to `datasets/mbb/` if a legacy copy appears).
`DO_NOT_ERASE/sr_school_slug_aliases.csv`	3	Panel / heuristic slug → corrected `sr_slug` when the crosswalk slug **404s** or otherwise fails; supports scrape-job generation without retyping URLs.
`DO_NOT_ERASE/bpm_player_season_raw.csv`	3	Long SR **advanced** scrape (many team-seasons); **expensive to re-fetch**—append/resume in scraper; lives with durable data, not disposable `mbb/` cache.
`DO_NOT_ERASE/bpm_player_season_matched.csv`	3	SR rows matched onto `(athlete_id, season, team_id)`; **costly to re-merge** at full raw scale—keep unless match logic or spine changes materially.

Out of scope for "team name" but same spirit: `DO_NOT_ERASE/draft_match_manual_feedback_template.csv` (Stage **2c** labels), `draft_name_dedupe_suggestions.csv` and similar under `mbb/` if you use them for human-in-the-loop name review—reuse when the spec calls for rerankers or suggestion round-trips.

Stage 1 — Spine from BOX data

We will implement spine refresh as **callable Python** (factored from today's `sdv_first` logic), invoked from `530` when needed:

- Ingests **sportsdataverse-style** MBB **player box**, **team box**, and **schedule** over an agreed season range.
- Persists tables under `datasets/mbb/` (and feathers / artifacts as we standardize) for draft match and panel build.

Grain: One row per player–season–team in box data, with `athlete_display_name`, `team_short_display_name`, etc., suitable for downstream matching and panel builds.

Stage 2 — Draft data and match to spine

We will implement draft linkage as **callable Python** (factored from `sdv_second`) so that NBA draft extract (e.g. `nbaplayersdraft.csv`) maps to `athlete_id` where the evidence supports it, and `athlete_id_draft_lookup.csv` remains what **530** uses for `Y_draft`. `530` runs this stage and then hits a **pause** for college / athlete QA if exports show misses.

2a College / school mapping (draft `college` ↔ ESPN team)

We will:

- Treat `DO_NOT_ERASE/college_aliases.csv` as the **primary** store of draft-college ↔ ESPN school string equity; match code loads it **before** fuzzy fallback and **merges** user edits from export/QA cycles back into this file (or a documented successor) rather than scattering one-off mappings only in memory.
- Use a controlled **alias dictionary** plus **home-built fuzzy** string matching (similarity floors/gaps tunable in code)—**not** a requirement to use the third-party `dedupe` package here.
- Emit **human-review** artifacts (unmapped college lists, ranked candidate schools) and iterate by **editing aliases** and re-running until coverage and quality are acceptable.
- Keep **school** as a **strong** constraint: wrong school is worse than noisy year.

2b Player name matching (draft `player` ↔ spine athletes)

We will use a **deliberate order of operations** (this replaces the old "narrow by school and year window first, then score names only inside that pool" approach):

1. **Name first (tractable at scale):** From the draft display name, build a **candidate set** on the spine using **name blocking** (e.g. surname / token buckets, or **top-K** fuzzy retrieval)—**never** a full naive cross-product of every draft row × every box row.
2. **Score names:** Apply a **home-built composite** (split surname/given + full-string fuzzy, normalization, generational suffixes, initials)—implemented with `rapidfuzz`-style ratios or equivalent—not a separate "exact-only pass then fuzzy pass"; identical normalized strings simply earn top scores in the same machinery.
3. **School second:** Require agreement with the draft row's **mapped** `team_id` / `team_short_display_name` among surviving candidates.
4. **Year last:** Use draft year vs spine collegiate timing **only after** name + school have narrowed the field—as **tie-break** or soft ordering, **not** as the primary hard filter that defines the initial candidate pool.

We will enforce **quality gates** (thresholds on composite, last-name, and full-string scores) before accepting an automatic `athlete_id`.

We will keep inspection exports (review CSVs, unmapped athlete candidates, diagnostics) so **CELL-level summaries** (or equivalent) report **draft-centric match rates**.

2c Human labels and overrides (planned)

We will treat structured `accept` / `reject` rows in `draft_match_manual_feedback_template.csv` as first-class input where the spec says so:

- **Minimum:** Labels feed an **optional** learned reranker (e.g. `sklearn` logistic regression on hand-crafted features) that only refreshes a **suggestions** file for review—unless we promote it.
- **Target:** `accept` rows (and/or a dedicated **override** file) **immediately** set `athlete_id` on the corresponding draft row before export, so human truth propagates to `athlete_id_draft_lookup` without hand-editing box scores.

(Exact file schema and merge order TBD as we finish this section.)

2d Match window / guards

We will support parameters such as **spine season range**, optional **minimum draft year** for automatic matching, and **consistent** application of those guards in both the matcher and eyeball helpers—documented here once values are chosen.

Milestone — "First summit": Spine + draft linkage sufficient for **530** to build a panel with `Y_draft` using **PPM** and/or **minutes** as `PERF_METRIC`.

Stage 3 — Sports Reference (SR) augmentation

We will expose SR work as **callable Python** (from `scrape_bpm` / `bpm_merge_to_530`), orchestrated by `530`:

- Scrape men's team-season **advanced** tables into a long **raw** file under `DO_NOT_ERASE` (current layout).
- **Reuse** `sr_school_slug_crosswalk.csv` and `DO_NOT_ERASE/sr_school_slug_aliases.csv` (see **Durable team / school string equity**) when building jobs and fixing failed slugs; refresh those CSVs only when **evidence** (404s, bad matches, QA) requires it—not on every spine refresh.
- Crosswalk ESPN schools to SR **slugs**, build scrape jobs, and **match** SR player-season rows onto `(athlete_id, season, team_id)` —name + slug + SR-year logic on the SR side, ESPN ids on the panel side.
- Produce `DO_NOT_ERASE/bpm_player_season_matched.csv` (raw scrape → `DO_NOT_ERASE/bpm_player_season_raw.csv`) that **530** left-merges when `PERF_METRIC` requires SR columns; `datasets/mbb/` holds lighter artifacts (e.g. `bpm_scrape_jobs.csv`, `bpm_panel_rows_unmatched.csv`).

530 may **pause** after scrape-job generation or after a partial scrape for you to fix slugs / aliases before continuing.

Milestone — "Second summit": 530 runs end-to-end with `PERF_METRIC` in **BPM / OBPM / DBPM** space (and compares shapes to PPM/minutes).

Stage 4 — 530 modeling panel (conductor + panel logic)

We will keep `530_sports_pipeline.ipynb` as the **conductor**: it **calls** Stages **1–3** via `sports_pipeline/` (stubs until ported), then **panel / EDA / inference** via `sports_pipeline/panel_build.py`. Use `obsolete_files/sports_gameplan_old/530_sports_pipeline_bkup.ipynb` only as a **parts bin** when moving logic out of the old cells.

We will make **530** (notebook + small panel layer) responsible for:

Piece	Intent
Grain	Player–season (aggregated from box as we define—consistent with memo).
`Y_draft`	**Ever-draft** v0 from `athlete_id_draft_lookup.csv` (refine to cohort logic later if pre-specified).
`perf` / `PERF_METRIC`	Configurable: **ppm**, **minutes**, **bpm**, **obpm**, **dbpm**, etc., with **fail-fast** or **two-pass** behavior when SR merge file is missing.
Pool quality	Leave-one-out teammate pool construct on `perf` (details as in decision memo / checklist).
EDA	At minimum: binned / ventile **draft rate** vs **pool quality** to read inverted-U shape.
Inference	Baseline **LPM** (or successor) with **pool quality** + quadratic term as appropriate; clustering if we keep that design.
Export	`player_season_panel_530.csv` (+ any sidecar diagnostics we lock in).

Exact cell order, winsorization, and robustness toggles will be written here or cross-linked from `obsolete_documents/sports_gameplan_old/College_Replication_Decision_Memo_Baseline_vs_Alternative_bkup.md` (baseline/memo archive) as we finalize.

Optional: `dedupe` for other projects

We may use `dedupe` later for **larger** entity-resolution problems: it learns weights over **multiple fields** with **blocking** and **saved training**. It does **not** encode a hard **name → school → year** cascade by itself—**our** outer code would enforce that. For **this** college pipeline, **home-built blocking + fuzzy + gates + overrides** is the default plan unless we explicitly add `dedupe`.

Fast path — first inverted-U checkpoint (dirty OK)

Purpose: Get a **read** on whether draft rate vs leave-one-out **pool quality** shows the hoped-for **nonlinear (inverted-U) shape** before investing in packaging, pause contracts, or matcher refinements. **One** scratch notebook or script is fine; salvage cells from `obsolete_files/sports_gameplan_old/530_sports_pipeline_bkup.ipynb` if that is faster than rewriting.

Rule: Do **not** delete or overwrite anything under `datasets/mbb/DO_NOT_ERASE/` while experimenting; write working copies under `datasets/mbb/` with a dated suffix if you need variants.

Checklist

- [ ] **1. Inputs present.** Confirm you can load: `datasets/mbb/mbb_df_player_box.csv` (spine), `datasets/mbb/athlete_id_draft_lookup.csv` (or equivalent for `Y_draft`), and—if you want SR-backed `perf`—`datasets/mbb/DO_NOT_ERASE/bpm_player_season_matched.csv`. If `player_season_panel_530.csv` already exists from an old run and you trust it, you may start there instead of rebuilding.
- [ ] **2. Lock v0 definitions (one short note in the notebook).** State: **grain** (player–season), **season range**, **minimum minutes** (or games) if any, `PERF_METRIC` for this run (`ppm`, `minutes`, or SR `bpm` / `obpm` / `dbpm` only if the merge file loads clean), and how `Y_draft` is coded (ever-draft v0 is enough).
- [ ] **3. Build `perf`** ** and `poolq_loo`.** Reuse logic from the **530** backup if possible: teammate leave-one-out pool quality on `perf` for each row. Skip winsorization at first if it slows you down; add a comment that it is "v0 raw."
- [ ] **4. Ventiles or bins.** Split `poolq_loo` into **ventiles** or **deciles**; within each bin, compute the **mean** or **share** of `Y_draft` and **N**. Plot bin midpoints vs draft outcome; overlay a simple smoother if you like. **Inverted-U** = draft prop **rises then falls** as pool quality increases (or your pre-specified direction).
- [ ] **5. Save advisor-facing artifacts.** Export the **figure** (PNG/PDF) with a **filename that includes the date**; write a **CSV snapshot** of the underlying binned table; keep **one** Markdown or notebook markdown cell with: paths used, row count, and `PERF_METRIC`. If the shape looks null or wrong-way, you still "win" the checkpoint—you learned early.
- [ ] **6. Optional same session: baseline LPM.** Regress `Y_draft` on `poolq_loo` + `poolq_sq` (and minimal controls if you already have them in the panel). Cluster-robust SE by player or team if copy-paste from old code is easy; otherwise OLS for a **dirty** sign check only.

After this checkpoint: If the pattern is promising, return to **Open points** and the clean conductor; if not, archive the scratch notebook and avoid polishing the full pipeline until the next design pivot.

Open points (negotiate next)

- **Package layout (baseline chosen):** `sports_pipeline/` package + root `pyproject.toml`; thin `scripts/` only for utilities. Further split/rename of stage modules as code grows.
- Precise **name-blocking** rule (buckets, **K**, minimum composite before school filter).
- **Year tie-break** rule when two candidates pass gates and share school.
- **Override** file format and apply order relative to automatic match.
- **530** aggregation recipes (minutes thresholds, seasons, winsorization) frozen into this doc or linked memo.
- **Pause contract:** which stages are **blocking** by default vs a `RUN_UNTIL=...` config flag.

Document history

- **2026-03-31:** Created from **SPORTS DATA Flow.docx**; iterative edits through March 2026.
- **2026-03-31: Pivot to forward-looking spec only**—removed "current code" retrospective; stages rewritten as **we will**; **530** called out for near–from-scratch rebuild; snapshots in `sports_gameplan_old` referenced as salvage sources.
- **2026-03-31: 530 as pipeline conductor** — target architecture: `.py` **modules** for ingest, draft match, SR scrape/merge; **530** notebook linear orchestration + explicit human **pause** points; legacy notebooks become migration sources.
- **2026-03-31: Clean slate at repo root** — removed `sdv_first`, `sdv_second`, `scrape_bpm`, `bpm_merge_to_530`, `530_sports_pipeline`, and related root backups/tests; `sports_gameplan_old` retains backup notebooks and `name_notebook_cells_bkup.py` for salvage.

- **2026-03-31: Companion specs archived** — `Scout_College_Basketball_Notebook_Workflow_How_To`, `Scout_BPM_Scraper_Spec`, `College_Replication_Decision_Memo_Baseline_vs_Alternative`, and `College_Replication_Run_Checklist_and_Decision_Gates` moved to `obsolete_documents/sports_gameplan_old/` with `_bkup` suffix (overwriting older same-named backups). `current_documents/sports_documents` now holds `SPORTS_DATA_GAMEPLAN.md`, `One_Page_Advisor_Brief_Template_College_Replication.md`, and **Vector_*** notes unless you relocate those later.
- **2026-03-31: Durable CSV mapping equity** — gameplan documents `college_aliases`, `sr_school_slug_crosswalk`, `sr_school_slug_aliases`, and points implementers at `DO_NOT_ERASE/README.txt`; stages **2a** and **3** call out **when** to load and **when** to extend those files.
- **2026-04-01: SR expensive artifacts in** `DO_NOT_ERASE` — `bpm_player_season_raw.csv`, `bpm_player_season_matched.csv`, `sr_school_slug_crosswalk.csv`, and schema backup `bpm_player_season_raw.csv.bak_before_fieldcount_repair` moved under `datasets/mbb/DO_NOT_ERASE/`; `old_scraped` (trash recovery) removed; `README.txt` updated.
- **2026-04-01: Conductor + package scaffold** — restored `530_sports_pipeline.ipynb` as the named conductor; added `sports_pipeline/` (`paths`, `config`, `ingest_box`, `draft_match`, `sr_bpm`, `panel_build`) and `pyproject.toml`. Stages 1–3 are **stubs** (file-exists checks); panel EDA loads `player_season_panel_530.csv` and matches prior ventile/LPM scratch behavior.
- **2026-04-01: Fast path checklist** — added **first inverted-U checkpoint** (dirty OK) so exploratory results precede packaging / open-point resolution.